

MODULE 7

Linux Basics & File System

Phase 2: Linux Fundamentals

Module Overview

Linux is the operating system of cybersecurity. It powers the vast majority of web servers, cloud infrastructure, network devices, and security tools used in the industry. Whether analyzing logs, running security assessments, managing servers, or responding to incidents, security professionals work in Linux environments daily. Phase 2 of this curriculum is entirely dedicated to building the Linux fluency that all subsequent modules depend on.

This module begins that journey by introducing the Linux operating system, its design philosophy, and its file system structure. Students will learn to navigate the terminal, understand how Linux organizes files and directories, and execute their first practical commands. Every concept is taught in the context of how it applies to cybersecurity work.

Lab Environment

All hands-on work in Phase 2 is performed on a Linux virtual machine. Students should have Ubuntu 22.04 LTS or Kali Linux running in VirtualBox or VMware before starting this module. If your VM is not yet configured, speak to your instructor all labs require a working Linux terminal.

Learning Objectives

By the end of this module, students will be able to:

- Explain what Linux is, why it matters for cybersecurity, and how it differs from Windows
- Navigate the Linux file system using the terminal with confidence
- Understand and describe the Linux Filesystem Hierarchy Standard (FHS) and the purpose of key directories
- Use essential navigation commands: `pwd`, `ls`, `cd`, `tree`
- Perform fundamental file and directory operations: `mkdir`, `touch`, `cp`, `mv`, `rm`, `rmdir`
- View and understand file contents using `cat`, `less`, `more`, `head`, and `tail`
- Explain the concept of absolute vs. relative paths and use both correctly
- Understand what hidden files are and how to display them
- Identify security-relevant directories and explain why they matter to a security analyst

Why Linux for Cybersecurity

Before touching a command, students should understand why Linux is the primary operating system for cybersecurity work. This context makes every command and concept more meaningful.

Reason	Explanation
It runs the internet	Over 90% of web servers, cloud platforms (AWS, Azure, GCP), and network infrastructure run Linux. Understanding Linux means understanding the systems you are defending and attacking.
Security tools are built for it	Kali Linux, the industry-standard penetration testing distribution, runs on Linux. Tools like Metasploit, Wireshark, Nmap, Burp Suite, and virtually every other security tool are Linux-native or Linux-first.
Transparency and control	Linux is open-source — every component can be inspected, modified, and understood. Security professionals can read the source code of the OS itself. This transparency is fundamental to trusted security work.
Granular permissions model	Linux's file permission and user/group model provides fine-grained access control. Understanding it is essential for configuring secure systems and detecting privilege escalation attacks.
Log-rich environment	Linux produces detailed logs for virtually every system event. Security monitoring, incident response, and forensic investigation all depend on the ability to read and analyse Linux logs.
It appears in all careers	SOC analysts, penetration testers, cloud security engineers, malware analysts, and incident responders all use Linux daily. It is non-negotiable knowledge for any cybersecurity career path.

Linux Distributions Relevant to Cybersecurity

Distribution	Purpose & Cybersecurity Use
Ubuntu	The most popular general-purpose Linux distribution. Widely used for servers, development environments, and learning. Phase 2 labs run on Ubuntu.
Kali Linux	Debian-based distribution pre-loaded with hundreds of security and penetration testing tools. The standard OS for ethical hacking and security assessments.
Parrot OS	A lighter alternative to Kali with similar security tooling. Popular for lower-spec hardware and privacy-focused work.
CentOS / RHEL	Enterprise Linux distributions used heavily in corporate server environments. Security analysts working in enterprise environments will encounter these.
Debian	The stable, community-driven distribution that underlies both Ubuntu and Kali. Valued for its security and reliability in server deployments.

The Linux Terminal

The terminal (also called the command line, shell, or console) is the primary interface for working with Linux in a professional context. While Linux has graphical desktops, security work especially remote server administration, scripting, and tool usage is almost entirely done through the terminal.

The Shell

The shell is the program that interprets and executes the commands you type. The most common shell in Linux is Bash (Bourne Again SHell). When you open a terminal, you are interacting with a Bash shell by default on most distributions.

```
user@kali:~$ # This is a typical Bash prompt
|           |           |
|           |           |_____ Current directory (~ means home)
|           |_____ Hostname (machine name)
|_____ Username
```

Prompt Element	Meaning
Username	The account currently logged in. The root user (superuser) prompt ends with # instead of \$
Hostname	The name of the machine — important when managing multiple servers via SSH
~	The tilde (~) is a shortcut representing the current user's home directory (/home/username)
\$	Indicates a regular (non-root) user. Root users see # — a critical visual cue about privilege level

Terminal Shortcut Reference

Tab = auto-complete commands and paths

Ctrl+C = cancel running command

Ctrl+L = clear

Up Arrow = previous command

Ctrl+R = search command history

Ctrl+A = move cursor to start of line

Ctrl+E = move cursor to end of line

The Linux Filesystem Hierarchy Standard (FHS)

Unlike Windows, which has separate drive letters (C:\, D:\), Linux has a single unified directory tree rooted at / (called the root directory). Everything files, devices, network sockets, processes is represented as a file somewhere in this tree. The Filesystem Hierarchy Standard (FHS) defines where different types of files should be stored.



Directory	Full Name	Purpose & Security Relevance
/	Root	The top of the entire filesystem tree. Everything starts here. Only root has write access.
/bin	Binaries	Essential user command binaries (ls, cp, mv, cat). Available to all users, required for basic system operation.
/boot	Boot	Bootloader files and the Linux kernel. Tampering here can prevent the system from starting - protect strictly.
/dev	Devices	Device files representing hardware (disks, terminals, random number generators). /dev/null discards all data written to it.
/etc	Et cetera / Config	System-wide configuration files. Critical for security: contains passwd, shadow, hosts, sudoers, SSH config, cron jobs. Frequently targeted by attackers.
/home	Home	Personal directories for each user (/home/alice, /home/bob). Contains user files, shell configs, and SSH keys.
/lib	Libraries	Shared libraries required by /bin and /sbin binaries. Similar to .dll files in Windows.
/opt	Optional	Third-party application software installed outside the standard package manager. Many security tools install here.
/proc	Processes	Virtual filesystem exposing kernel and process information in real time. /proc/[PID]/ contains information about each running process.
/root	Root Home	The home directory of the root (superuser) account. Separate from /home - accessible only to root.
/sbin	System Binaries	System administration binaries (ifconfig, iptables, reboot). Generally only used by root.
/srv	Service Data	Data served by the system (web server files, FTP data). What you put in /var/www/html for web servers.
/tmp	Temporary	Temporary files deleted on reboot. Writable by all users - attackers often stage files here. Monitor for unexpected executables.
/usr	User	Secondary hierarchy for user applications, documentation, and libraries. /usr/bin contains most user commands.

<code>/var</code>	Variable	Variable data that changes during operation: logs (<code>/var/log</code>), mail spools, databases, caches. The most security-relevant directory for log analysis.
-------------------	----------	---

Security-Critical Directories

Certain directories deserve special attention from a security analyst perspective either because they contain sensitive configuration data, logs that reveal system activity, or because they are commonly abused by attackers.

`/etc`: The Security Analyst's First Stop

The `/etc` directory contains nearly all system-wide configuration files. Understanding what lives here is essential for both hardening systems and investigating incidents.

File	Security Significance
<code>/etc/passwd</code>	Lists all user accounts: username, UID, GID, home directory, and default shell. World-readable - attackers enumerate this to discover usernames for targeted attacks. Passwords are NOT stored here in modern systems.
<code>/etc/shadow</code>	Contains hashed passwords for all user accounts. Readable only by root. Attackers who gain root access target this file to crack password hashes offline.
<code>/etc/group</code>	Defines groups and their members. Auditing group membership reveals which users have elevated access (sudo group, docker group, etc.).
<code>/etc/sudoers</code>	Controls which users can run commands as root using sudo. Misconfigured sudoers rules are a major privilege escalation vector.
<code>/etc/hosts</code>	Local DNS resolution table mapping hostnames to IPs. Malware modifies this file to redirect legitimate domains to malicious IPs.
<code>/etc/crontab</code>	System-wide scheduled tasks. Attackers add malicious cron jobs here to maintain persistence after a compromise.
<code>/etc/ssh/sshd_config</code>	SSH server configuration. Misconfigurations (root login enabled, weak ciphers, no key-only auth) create critical remote access vulnerabilities.
<code>/etc/fstab</code>	Defines how disk partitions are mounted. Misconfiguration can expose writable mounts or bypass security controls.

/var/log: The Security Analyst's Evidence Trail

Log files in /var/log are the primary source of evidence during incident investigation and security monitoring. Every security analyst must know what each log file records.

Log File	What It Records
<code>/var/log/auth.log</code>	Authentication events: all login attempts (successful and failed), sudo usage, SSH sessions, and su commands. First place to look during a breach investigation.
<code>/var/log/syslog</code>	General system messages from the kernel and services. Useful for detecting service crashes, hardware errors, and unexpected system behaviour.
<code>/var/log/kern.log</code>	Kernel-specific messages including hardware errors, driver issues, and kernel panics.
<code>/var/log/apache2/</code>	Apache web server logs: access.log (all HTTP requests) and error.log. Essential for detecting web attacks, directory traversal, and injection attempts.
<code>/var/log/nginx/</code>	Nginx web server logs. Same purpose as Apache logs for Nginx-served sites.
<code>/var/log/dpkg.log</code>	Package installation and removal history. Reveals when software was installed - useful for detecting attacker-installed backdoors.
<code>/var/log/wtmp</code>	Binary log of all logins, logouts, and system reboots. Read with the last command.
<code>/var/log/faillog</code>	Records failed login attempts per user. Read with the faillog command.
<code>/var/log/cron</code>	Records scheduled task (cron job) execution. Reviewing this reveals if unauthorised scheduled tasks are running.

Absolute vs. Relative Paths

Every file and directory in Linux has an address called a path. Understanding the difference between absolute and relative paths is fundamental to navigating the filesystem confidently.

Path Type	Explanation & Example
Absolute Path	Starts from the root directory / and gives the complete address of a file regardless of where you currently are. Always begins with /. Example: /home/alice/documents/report.txt - this path is unambiguous from anywhere in the system.
Relative Path	Describes a location relative to your current working directory. Does NOT start with /. Example: if you are in /home/alice, then documents/report.txt refers to the same file without specifying the full path.

Special Path Symbols:

Symbol	Meaning & Example
.	Current directory. <code>cd .</code> stays where you are. <code>cp file.txt .</code> copies file.txt to the current directory.
..	Parent directory (one level up). <code>cd ..</code> moves up one level. <code>cd ../../etc</code> goes up two levels then into etc.
~	Current user's home directory. <code>cd ~</code> always takes you home. <code>~/Desktop</code> is shorthand for <code>/home/username/Desktop</code> .
-	Previous directory. <code>cd -</code> switches back to the last directory you were in - useful when working between two locations.

```
$ pwd    # Where am I right now?
/home/alice
$ cd /var/log    # Absolute path - go to /var/log from anywhere
$ cd ../../home/alice    # Relative path - up two levels, then into /home/alice
$ cd ~    # Always returns to home directory
$ cd -    # Return to previous directory (/var/log)
```

Navigation Commands

Pwd: Print Working Directory

`pwd` tells you exactly where you are in the filesystem. Always run `pwd` when you are unsure of your current location - especially important when working with absolute and relative paths.

```
$ pwd
/home/alice
$ cd /etc/ssh
$ pwd
/etc/ssh
```

Ls: List Directory Contents

Ls lists the files and directories in a location. It is one of the most frequently used commands in Linux. Understanding its options is essential for daily work.

Command	What It Does
<code>ls</code>	List files and directories in the current directory (basic output, no hidden files)
<code>ls -l</code>	Long listing format - shows permissions, ownership, file size, and modification date for each item
<code>ls -a</code>	Show ALL files including hidden files (those whose names begin with a dot, e.g. <code>.bashrc</code>)
<code>ls -la</code>	Combine long format and all files - the most informative and commonly used ls variant
<code>ls -lh</code>	Long format with human-readable file sizes (KB, MB, GB instead of raw bytes)
<code>ls -R</code>	Recursive listing - lists all files in the current directory and all subdirectories
<code>ls -lt</code>	Long format sorted by modification time (newest first) - useful for spotting recently changed files
<code>ls /etc</code>	List contents of a specific directory without changing into it

```
$ ls -la /etc # Long listing of /etc including hidden files
total 876
drwxr-xr-x 128 root root 4096 Apr 28 09:15 .
drwxr-xr-x 20 root root 4096 Apr 20 08:00 ..
-rw-r--r-- 1 root root 2981 Mar 14 11:22 passwd
-rw-r----- 1 root shadow 1423 Mar 14 11:22 shadow
drwxr-xr-x 2 root root 4096 Feb 10 15:30 ssh
-rw-r--r-- 1 root root 552 Jan 05 10:00 hosts
```

Reading ls -l Output:

<code>-rw-r--r--</code>	<code>1</code>	<code>root</code>	<code>root</code>	<code>2981</code>	<code>Mar 14 11:22</code>	<code>passwd</code>
Permissions	Links	Owner	Group	Size	Modified	Filename

cd: Change Directory

cd moves you to a different location in the filesystem. Mastering cd with both absolute and relative paths is the foundation of terminal navigation.

Command	What It Does
<code>cd /var/log</code>	Navigate to /var/log using an absolute path
<code>cd Documents</code>	Navigate into the Documents subdirectory of the current directory (relative path)
<code>cd ..</code>	Move up one directory level to the parent
<code>cd ../../</code>	Move up two directory levels
<code>cd ~</code>	Return to the current user's home directory
<code>cd -</code>	Switch to the previous directory
<code>cd /</code>	Navigate to the root of the filesystem

tree: Visual Directory Structure

tree displays the directory structure as a visual tree diagram. Extremely useful for understanding the layout of a directory at a glance.

May need to be installed: `sudo apt install tree`.

```
$ tree /etc/ssh
/etc/ssh
├── moduli
├── ssh_config
├── sshd_config
├── ssh_host_ecdsa_key
├── ssh_host_ecdsa_key.pub
├── ssh_host_ed25519_key
└── ssh_host_rsa_key

$ tree -L 2 /var    # Show tree 2 levels deep
$ tree -a ~        # Show all files including hidden
```

File & Directory Operations

mkdir: Make Directory

mkdir creates new directories. Use it to organize your work into a structured folder hierarchy.

Command	What It Does
<code>mkdir reports</code>	Create a directory named 'reports' in the current location
<code>mkdir /tmp/investigation</code>	Create a directory at an absolute path
<code>mkdir -p logs/2024/april</code>	Create nested directories in one command (-p creates parents as needed)
<code>mkdir -m 700 private</code>	Create a directory with specific permissions (700 = owner only)

```
$ mkdir -p ~/investigations/case001/logs # Create full nested path at once
$ ls ~/investigations/case001/
logs
```

touch: Create Empty File / Update Timestamp

touch creates a new empty file if the filename does not exist, or updates the access and modification timestamps of an existing file. Commonly used to quickly create placeholder files.

```
$ touch notes.txt # Create an empty file called notes.txt
$ touch report.md evidence.csv # Create multiple files at once
$ ls -l notes.txt
-rw-rw-r-- 1 alice alice 0 Apr 28 10:00 notes.txt
```

cp: Copy Files and Directories

cp copies files or directories from one location to another. The original is preserved.

Command	What It Does
<code>cp file.txt backup.txt</code>	Copy file.txt to backup.txt in the same directory
<code>cp file.txt /tmp/</code>	Copy file.txt into the /tmp/ directory
<code>cp file.txt /tmp/newname.txt</code>	Copy and rename in one step
<code>cp -r mydir/ /tmp/mydir_backup/</code>	Copy a directory and all its contents recursively (-r is required for directories)

<code>cp -p file.txt backup.txt</code>	Preserve original file metadata: permissions, timestamps, and ownership
<code>cp -i file.txt existing.txt</code>	Interactive mode - prompt before overwriting an existing file

mv: Move or Rename

mv moves a file or directory to a new location. When source and destination are in the same directory, it acts as a rename. Unlike cp, mv removes the file from its original location.

Command	What It Does
<code>mv oldname.txt newname.txt</code>	Rename a file in the same directory
<code>mv file.txt /var/log/</code>	Move file.txt into /var/log/
<code>mv file.txt /var/log/new.txt</code>	Move and rename simultaneously
<code>mv -i file.txt destination/</code>	Prompt before overwriting an existing file
<code>mv dir1/ /tmp/dir1_moved/</code>	Move an entire directory

rm: Remove Files

rm permanently deletes files. Linux has no recycle bin deleted files cannot be recovered with standard tools. Use with care, especially with the -r and -f flags.

Command	What It Does
<code>rm file.txt</code>	Delete a single file (asks nothing - file is gone immediately)
<code>rm -i file.txt</code>	Interactive mode - prompts for confirmation before deleting (recommended habit)
<code>rm -r directory/</code>	Recursively delete a directory and all its contents
<code>rm -f file.txt</code>	Force delete without prompting, ignoring errors

Warning: rm -rf Is Irreversible

The command `rm -rf /path/to/directory/` deletes everything in that path instantly with no confirmation and no recovery. Never run `rm -rf` with a path containing variables or wildcards unless you are absolutely certain of what will be deleted. A misplaced space in `rm -rf /` would attempt to delete the entire filesystem. Always double-check the path before executing `rm -r`.

Viewing File Contents

Reading file contents is one of the most common tasks for security analysts examining configuration files, log entries, scripts, and evidence. Linux provides several commands for viewing files, each suited to different situations.

cat: Print File Contents

cat (concatenate) outputs the entire contents of a file to the terminal at once. Best for short files. For large files, use less or tail instead.

Command	What It Does
<code>cat filename.txt</code>	Display entire file contents
<code>cat -n filename.txt</code>	Display file with line numbers useful for referencing specific lines in reports
<code>cat file1.txt file2.txt</code>	Concatenate and display multiple files one after another
<code>cat /etc/passwd</code>	View all user accounts a common security analyst task
<code>cat /etc/hosts</code>	View local DNS hostname mappings

```
$ cat -n /etc/hosts
 1      127.0.0.1    localhost
 2      127.0.1.1    ubuntu
 3      ::1          localhost ip6-localhost ip6-loopback
 4      ff02::1      ip6-allnodes
```

less and more: Paginated Viewing

For large files such as log files with thousands of lines cat floods the terminal and is unusable. less and more display content one screen at a time, allowing you to scroll and search.

Command / Key	What It Does
<code>less filename</code>	Open file in scrollable viewer. Better than more allows both forward and backward navigation
<code>more filename</code>	Open file for forward-only paginated viewing (older, more limited than less)
Space	Scroll down one full screen
b	Scroll back up one screen (less only)
/ search_term	Search forward for a term within the file (e.g. /ERROR to find all error entries)
n	Jump to the next match after a search

N	Jump to the previous match
g	Go to the beginning of the file
G	Go to the end of the file
q	Quit and return to the terminal

```
$ less /var/log/auth.log # View auth log one page at a time
$ less /var/log/syslog
```

head and tail — View File Beginning or End

head displays the first lines of a file; tail displays the last lines. For security analysts, tail is especially important because log files grow from the bottom — new entries are always at the end.

Command	What It Does
<code>head filename</code>	Show the first 10 lines of a file (default)
<code>head -n 20 filename</code>	Show the first 20 lines
<code>tail filename</code>	Show the last 10 lines (most recent log entries)
<code>tail -n 50 filename</code>	Show the last 50 lines
<code>tail -f filename</code>	Follow mode — stream new lines as they are added in real time. Essential for live log monitoring.
<code>tail -f /var/log/auth.log</code>	Watch authentication events as they happen — useful during active incident investigation

```
$ tail -n 20 /var/log/auth.log # Last 20 authentication events
Apr 28 09:44:22 ubuntu sshd[1423]: Failed password for root from
192.168.1.105 port 52341 ssh2
Apr 28 09:44:24 ubuntu sshd[1423]: Failed password for root from
192.168.1.105 port 52342 ssh2
Apr 28 09:44:26 ubuntu sshd[1423]: Failed password for root from
192.168.1.105 port 52343 ssh2
Apr 28 09:44:28 ubuntu sshd[1424]: Accepted publickey for alice from
10.0.0.5 port 55100 ssh2
```

Security Analyst Tip: Spotting Brute Force in Auth Logs

The example output above shows three consecutive 'Failed password for root' messages from the same IP within 6 seconds — a classic brute force SSH pattern. A security analyst sees this and immediately investigates: Is the source IP internal or external? How many attempts were there in total? Did any succeed? Module 12 covers log analysis in depth — this example previews why reading log files is a critical daily skill.

Hidden Files & Dot Files

In Linux, any file or directory whose name begins with a dot (.) is hidden by default — it will not appear in a standard `ls` listing. These are called dot files or dotfiles. They are not hidden for security purposes — they are hidden to keep configuration files out of everyday directory listings.

```
$ ls ~      # Regular listing - hidden files not shown
Desktop Documents Downloads Music Pictures
$ ls -a ~   # All files - hidden files now visible
.  .. .bash_history .bash_logout .bashrc .profile .ssh
Desktop Documents Downloads Music Pictures
```

Hidden File	Security Significance
<code>.bash_history</code>	Records every command the user has typed. A goldmine for forensic investigators — reveals exactly what an attacker or user did during a session.
<code>.bashrc</code>	Bash configuration file executed on every interactive shell start. Attackers add malicious commands here to establish persistence.
<code>.bash_profile</code>	Executed on login. Like <code>.bashrc</code> , can be modified for persistence by attackers.
<code>.ssh/</code>	Directory containing SSH keys and the <code>authorized_keys</code> file. Attackers add their public key to <code>authorized_keys</code> to maintain persistent SSH access.
<code>.ssh/authorized_keys</code>	Contains public keys allowed to authenticate via SSH without a password. Check this file during incident response for unauthorised keys.
<code>.profile</code>	General shell initialisation file. Can contain environment variable settings and startup commands.

Linux vs. Windows — Key Differences

Many students transitioning to Linux come from a Windows background. Understanding the fundamental differences prevents confusion and accelerates learning.

Aspect	Linux	Windows
File separator	<code>/var/log/syslog</code>	<code>C:\Windows\System32\drivers</code>
Case sensitivity	Case-sensitive: <code>File.txt</code> $\neq$ <code>file.txt</code> $\neq$ <code>FILE.TXT</code>	Case-insensitive: All three refer to the same file

User interface	Terminal-primary; GUI optional	GUI-primary; PowerShell/CMD secondary
Root/Admin	root user (UID 0)	Administrator account
Software install	Package manager (apt, yum, dnf)	Installer executables (.exe, .msi)
Config files	Plain text files in /etc	Registry + XML/INI files in Program Files
Permissions	User/Group/Other rwx model	ACL-based NTFS permissions
File extensions	Not required — executability set by permission bit	Required (.exe, .bat, .ps1 determine execution)
Path separator	Forward slash: /	Backslash: \
Superuser access	sudo command for temporary elevation	Run as Administrator

Module 7 Command Quick Reference

Command	Syntax Example	What It Does
pwd	pwd	Print current working directory
ls	ls -la /var/log	List directory contents (long + hidden)
cd	cd /etc/ssh	Change to specified directory
tree	tree -L 2 /etc	Show directory structure as visual tree
mkdir	mkdir -p logs/case001	Create directory (and parents if needed)
touch	touch notes.txt	Create empty file or update timestamp
cp	cp -r src/ /tmp/backup/	Copy file or directory
mv	mv oldname.txt newname.txt	Move or rename file/directory
rm	rm -i file.txt	Remove file (interactive mode)
cat	cat -n /etc/passwd	Display file contents with line numbers
less	less /var/log/syslog	View large file with scrolling
head	head -n 20 /var/log/auth.log	Display first N lines of file
tail	tail -f /var/log/auth.log	Display last lines; -f follows live updates